

Investigacion modulos 5G

Módulo que se nos enseñó:

<https://www.tiendatec.es/maker-zone/comunicaciones-redes/2501-quectel-rm520n-gl-modulo-5g-iot-global-m2-8472496030328.html>

Buscamos que trabaje en banda de frecuencia n40
Opera en el rango de 2.300 MHz a 2.400 MHz (2,3 GHz).

Utiliza tecnología TDD (Duplexación por División de Tiempo)

En vez de hacerlo por frecuencia donde se utilizan dos canales separados para emitir y recibir en este caso simplemente transmitimos en un intervalo de tiempo y cedemos a misma frecuencia/canal recibir en el siguiente, dado que tenemos una velocidad muy alta a la hora de conmutar y por lo tanto se aprecia como un proceso continuo y fluido

De esta forma podemos tener regulación sobre los enlaces y poder elegir si queremos dar mas velocidad a la transmision o recepcion de datos otorgando un intervalo de tiempo mas prolongado a una que otra

Si es cierto que esta tecnologia se usa mas en transmision asimetrica a diferencia de la division por frecuencia donde se usa mas para navegar por internet dada la tendencia asimetrica donde suele interesar mas descargar

5G SA Sub-6: descarga 2.4 Gbps; carga 900 Mbps
5G NSA Sub-6: descarga 3.4 Gbps; carga 550 Mbps

5G NSA Sub-6 descarga mas rapido aun utilizando una infraestructura 4G LTE en vez de una red propiamente hecha para el 5G como es SA?

ejemplo dongle: MODEM DONGLE 5G CON INTERFAZ M.2 - USB 3.2

Importante en este caso no contamos con conexiones PCIe si no que tenemos que valernos con M2

Kit completo con base, antenas pcb internas

<https://sixfab.com/product/sixfab-5g-modem-kit-for-raspberry-pi-5/?srsltid=AfmBOooLmo2vbQNukCwkVIA1W7eYV-JKoEXO9Wjm3POZhix9aK-zfcWt>

Competidor directo del modulo que nos enseñaron:

https://www.dfrobot.com/product-2635.html?gad_source=1&gad_campaignid=23447358446&gbraid=0AAAAADucPIB7ir7H_Aj1NGDTup3ph0q7c&gclid=Cj0KCQjw_b_QBhCSARIsAP6hR4dWrpfnEBaUkKYB18PEK035T1204pfTOBi8IFUwproXxx7WrpJaLMaAIHPEALw_wcB

Más barato mismas prestaciones aparentemente y pilla banda de freq n40

Release 16

FM160-GL NO TIENE N40

https://es.aliexpress.com/item/1005010623741621.html?src=google&src=google&albch=shopping&acnt=439-079-4345&isdl=y&slnk=&plac=&mtctp=&albbt=Google_7_shopping&aff_platform=google&aff_short_key=UneMJZVf&gclsrc=aw.ds&albagn=888888&ds_e_adid=&ds_e_matchtype=&ds_e_device=c&ds_e_network=x&ds_e_product_group_id=&ds_e_product_id=es1005010623741621&ds_e_product_merchant_id=5551310678&ds_e_product_country=ES&ds_e_product_language=es&ds_e_product_channel=online&ds_e_product_store_id=&ds_url_v=2&albcpr=21116729971&albag=&isSmbAutoCall=false&needSmbHouyi=false&gad_source=1&gad_campaignid=21106574823&gbraid=0AAAAACbpfvacwp67X3YusXRYS1_D8y3ST&gclid=Cj0KCQjw_b_QBhCSARIsAP6hR4e20hzSqfX9zrnCX-y45pjC0N-J093a0leC84z3chLMOLa5D5s9C5EaAi9yEALw_wcB

Muy barato pero no acepta QMI ni MBIM no utiliza un chip Qualcomm Snapdragon X55 es release 15 y usa unisoc

Conclusión

Se pueden encontrar opciones algo mas baratas pero siempre haciendo un pequeño downgrade de prestaciones. Interesante el competidor directo mencionado

Planteamiento ALGORITMO

Contexto principal

Ahora mismo dependemos de un único dron así que lógicamente empezamos por establecer nuestros requisitos para declarar una flota de estos con diferentes specs.

Tareas a realizar:

Dejar de declarar como Dron1 mediante una ctt

En la db de operaciones necesito establecer además de mis pedidos y clientes una serie de datos en relación al estado de la flota

Mirar para establecer conexión al unísono en mas de un puerto ya que ahora solo uso por tcp el 5763

Terminar de establecer las rutas (añadir niveles de vuelo? 5m por nivel?)

FLOTA DRONES

Nos encontramos con una flota de drones heterogénea ya que tendremos distintas categorías de drones y por tanto specs(vease la capacidad de carga, *velocidad*, autonomía)

Por curiosidad si se planteara un problema con enrutamientos mas complejos se han encontrado las siguientes soluciones:

Dada esta situación nos encontramos con un problema ya definido que es https://es.wikipedia.org/wiki/Problema_de_rutas_de_veh%C3%ADculos con ciertos parametros que supondrán ciertos costes a la hora de decidir que vehiculo se asigna a que pedido.

Es por eso que tenemos que tener en cuenta las soluciones para estos use-cases concretos como CVRP (Capacitated Vehicle Routing Problem) donde aseguramos que no se exceda la capacidad de carga del dron.

<https://github.com/darylGan/HFVRP>

Muy importante igual que la capacidad por peso tener en cuenta que tratamos con vehículos eléctricos por lo tanto a diferencia de un modelo tradicional hay que tener en cuenta que tenemos una autonomía limitada y la necesidad de hacer paradas programadas en estaciones de carga <https://github.com/NeiH4207/EVRP-Python>

Ya que nos desaparece el problema de enrutamiento dado que tenemos rutas preestablecidas de ida y vuelta, lo único en lo que tenemos que centrarnos es en un problema de asignación

Como posible solución he encontrado para el problema de asignación la de Jonker-Volgenant., este algoritmo se basa en una matriz de costes para que dados una serie de drones(dron1,dron2,etc...) se les asigne un coste para realizar una tarea, en este caso un pedido. Si por ejemplo un dron tiene una capacidad de carga inferior al peso del pedido o un nivel de batería inferior al necesario para el viaje de ida y vuelta su coste se establecerá en infinito con tal de descartar que nos pueda crear una pareja entre ese dron y pedido

Gracias a la telemetría tendremos acceso siempre a la carga actual del dron pero no sabremos si dado su nivel de batería tendrá suficiente para realizar un pedido en concreto. Es por eso que necesitamos una forma de modelar esta situación.

Se considerará un modelización lineal para simplificar el problema, que podría ser:

$$Eida(\%) = \alpha \cdot t_{ida} + \beta \cdot m_{payload} \cdot d_{ida} + (\gamma \cdot h?)$$

$$Evuelta(\%) = \alpha \cdot t_{vuelta} \cdot (\gamma \cdot h?)$$

$$Eviaje(\%) = Eida + Evuelta$$

Tener en cuenta que $t_{ida} = t_{vuelta}$ además de que $t = \frac{d}{V_{crucero}(dronx)}$

Para por ejemplo establecer coste infinito a que un dron x realice una tarea y entonces deberíamos plantear de la siguiente manera:

Para tener margen de seguridad vamos a obligar a que para que un dron esté obligado a tener un 20% de su capacidad de carga máxima como reserva

Así pues procedería de la siguiente forma:

$$Esobrante(\%) = Eactual - Eviaje$$

$$Si \text{ Esobrante}(\%) \geq 20$$

Carga dron

$$tcarga = \frac{Enecesaria - Eactual}{Pcargador}$$

Quitando estas hardconstraints de carga y energía tenemos que tener en cuenta que podemos ver costes más allá por ejemplo

Si tenemos dos drones al 100% para un pedido de 1Kg, uno con capacidad de 1Kg y otro de 4Kg, podemos asignar que el de 4Kg tenga más coste que el de 1Kg ya que sería gastar más energía y además si saliera un pedido de 4Kg después perdemos la oportunidad de mandarlo

También en casos donde los dos tengan suficiente batería pero uno muy cerca del umbral del 20%, es preferible que este tenga más coste en realizar el pedido ya que si fuera el del límite de batería volvería con muy poca y habría que esperar mucho tiempo en que se cargará y tuviera la suficiente como para reincorporarse, mientras que si va el que tiene mucha el otro podrá cargarse para realizar un pedido a futuro sin quedarse al límite y el que vuelve tendrá la suficiente como para realizar otro pedido posiblemente

A su vez también podemos tener en cuenta el enviar drones más rápidos o lentos según el pedido

FUNCION DE COSTES:

$$C_{i,j} = w1 \cdot (E_{viaje}) + w2 \cdot E_{equilibrioBateria} + w3 \cdot (Payload_{max} - peso_{paquete})$$

Se podría añadir un 4º término $w4$ multiplicado por el tiempo de espera
 $w1$ lo usamos para normalizar valores

```
from scipy.optimize import linear_sum_assignment
```

Si es cierto que podemos usar el algoritmo greedy/voraz y hacer iterativo además de que técnicamente tiene menos complejidad, pero estaríamos haciendo iterativo sobre cada pedido sin mirar el siguiente y si por ejemplo tenemos dron pesado y dron pequeño y ambos pueden llevar el pedido, y tenemos pedido1 a 1km y pedido 2 a 10km, el algoritmo lee el pedido1 y envía directamente al dron óptimo que sería el pequeño para gastar menos pero cuando llega al pedido2 se ve obligado a enviar el dron más grande a un pedido a 10km. Además requiere de más código mientras que en el algoritmo usamos una librería y solo tenemos que centrarnos en modelar correctamente la función de costes

Me gustaría hacer una comparación con un algoritmo greedy y ver si conseguimos optimizar con Jonker-Volgenant.